MOBILE APPLICATION AND DEVELOPMENT

Basic UI Components & Handling User Input

Show things on screen, then make them respond

Ekramul Haque Amin · Lecturer, AI & Data Science · Dhaka International University

TODAY'S ROADMAP

What We Will Cover

1. The building blocks: TextView, EditText, Button and ImageView
2. Choice widgets: CheckBox, RadioButton and Switch
3. Click listeners: making a button do something
4. Reading input, validating it, and giving feedback
5. Toast, Snackbar, other events and ViewBinding
6. Putting it together in a Greetings app

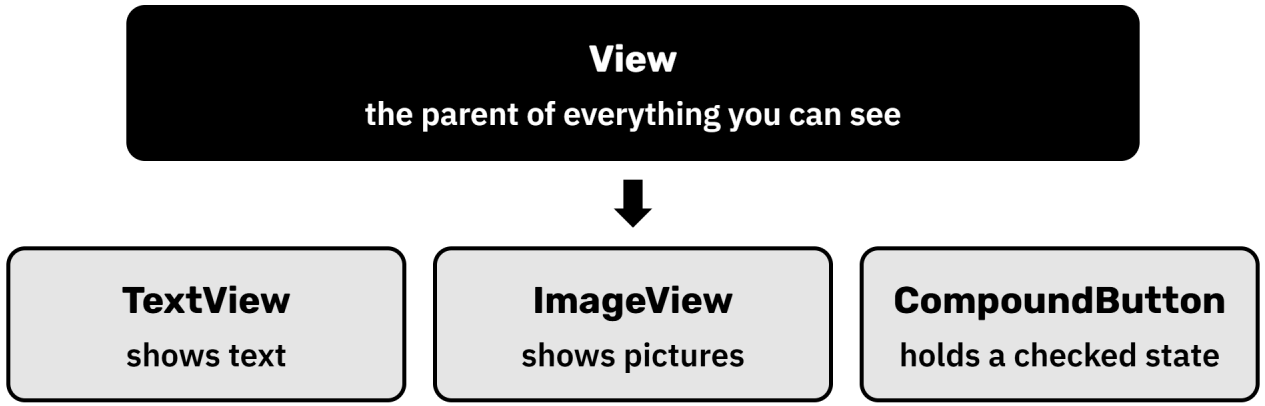
DID YOU KNOW

Every View on screen is a child of the same Java class: View

TextView, Button, EditText and ImageView are all just different kinds of View.

THE FAMILY TREE

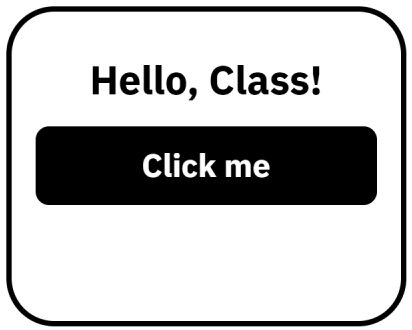
Everything Is a View



> Button and EditText extend TextView. CheckBox, RadioButton and Switch extend CompoundButton.

TWO WORLDS

XML Draws It, Java Controls It



XML: what it looks like

```
TextView tv = findViewById(R.id.tv);
tv.setText("Welcome!");
```

Java: what it does

> Each widget gets an android:id in XML so Java can find it.

QUICK RECAP

The Big Picture

- Every widget is a View
- XML describes the look, Java adds the behaviour
- The android:id is the link between the two

> Next up: TextView, the widget that shows text.

WIDGET 1

TextView

Show text on the screen

Fun fact: a Button is a TextView with a background and a click effect.

THE BASICS

A TextView in XML

```
<TextView
    android:id="@+id/tvTitle"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Hello, Android!"
    android:textSize="24sp"
    android:textColor="#000000" />
```

> **Use sp for text sizes so they follow the user’s font setting. Use dp for everything else.**

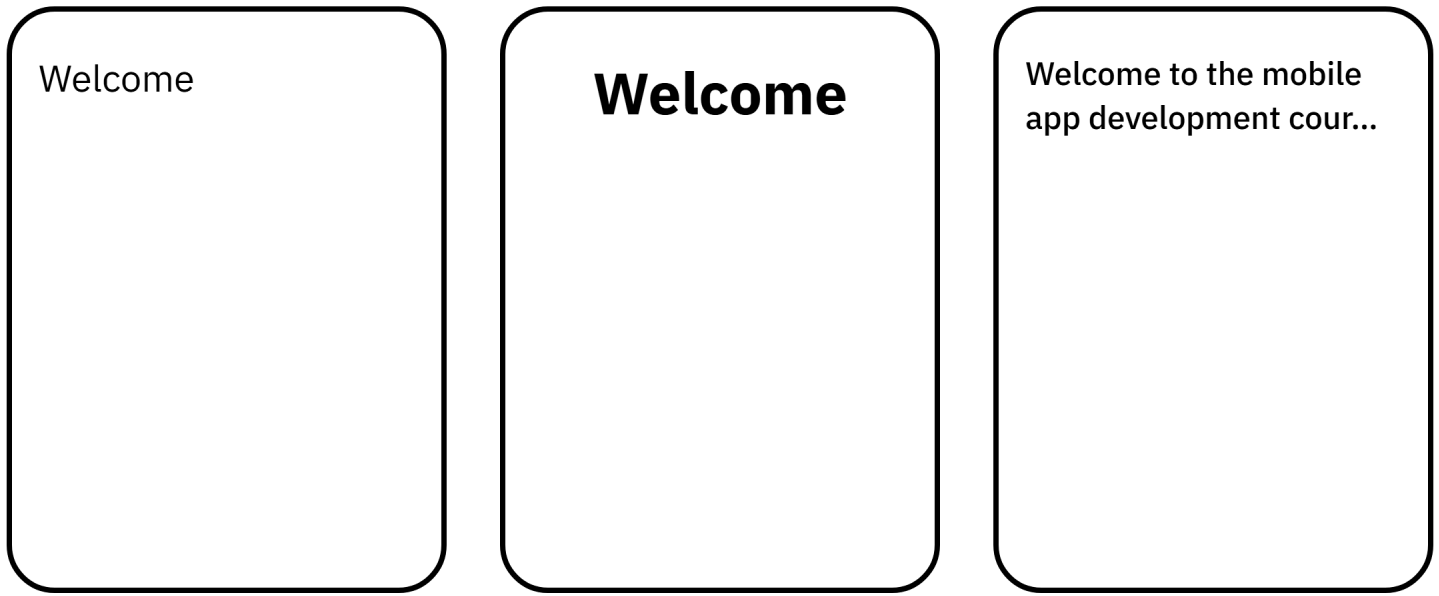
MAKE IT YOURS

Common TextView Attributes

Attribute	What it does
android:textSize	Size, in sp
android:textColor	Colour of the text
android:textStyle	bold, italic or normal
android:fontFamily	Pick a font
android:gravity	Align text inside the view
android:maxLines	Limit how many lines show
android:ellipsize	Add … when text is cut off

COMPARE

Same Text, Different Attributes



FROM JAVA

Changing Text While the App Runs

```
TextView tvTitle = findViewById(R.id.tvTitle);

tvTitle.setText("Welcome back!");
tvTitle.setTextSize(28);
tvTitle.setTextColor(Color.BLACK);
tvTitle.setVisibility(View.VISIBLE);
```

> **Every XML attribute has a matching setter. Learn the pattern: android:text becomes setText().**

GOOD HABIT

Do Not Hard-Code Strings

```
android:text="Welcome"
```

Avoid

```
android:text="@string/welcome"
```

Better

```
<!-- res/values/strings.xml -->
<string name="welcome">Welcome</string>
```

> **Strings in one file are easy to change and easy to translate into Bangla.**

QUICK RECAP

TextView

- Shows text, set by XML or setText()
- Use sp for text size, dp for sizes
- Put text in strings.xml, not in the layout

> Next up: EditText, where the user types.

WIDGET 2

EditText

Let the user type

Fun fact: choosing the right inputType makes Android show the right keyboard automatically.

THE BASICS

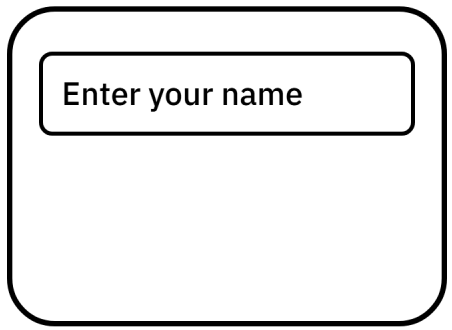
An EditText in XML

```
<EditText
    android:id="@+id/etName"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Enter your name"
    android:inputType="textPersonName" />
```

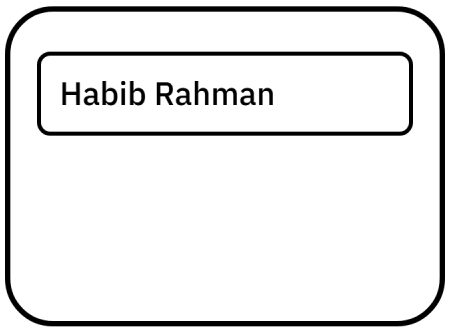
> **A hint is grey placeholder text that disappears when the user starts typing.**

WHAT IT LOOKS LIKE

Hint, Then Typed Text



Empty: hint shows



Typing: hint is replaced

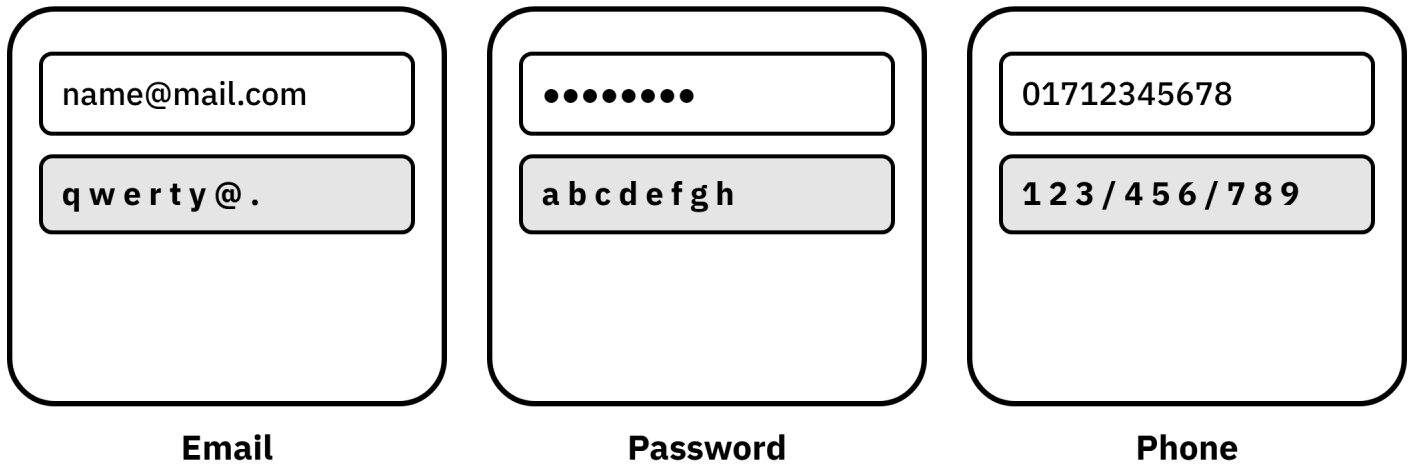
PICK THE KEYBOARD

inputType Changes Everything

inputType	The user gets
text	Normal letters keyboard
textEmailAddress	Keyboard with @ and .
textPassword	Dots instead of characters
number	Digits only
phone	Dial pad
textMultiLine	Enter makes a new line

SEE THE DIFFERENCE

Different Fields, Different Keyboards



POLISH IT

More EditText Attributes

android:maxLength

Stops typing after a set number of characters

android:lines

Sets how many lines tall the box is

android:imeOptions

Changes the keyboard’s action key, such as Done or Next

android:drawableStart

Puts a small icon inside the field

QUICK RECAP

EditText

- hint gives a clue, inputType picks the keyboard
- maxLength and imeOptions improve the experience
- Reading the value comes later, with click listeners

> Next up: Button.

WIDGET 3

Button

The thing everyone taps

Fun fact: a Button needs no extra code to look pressed. Android adds the ripple effect for you.

THE BASICS

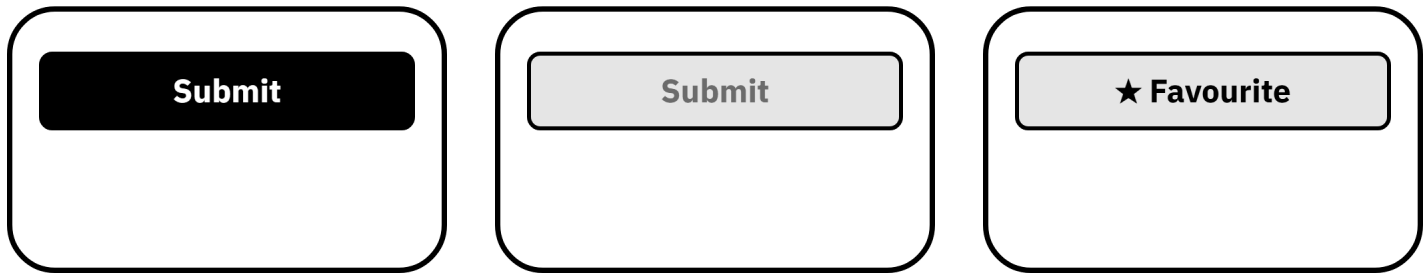
A Button in XML

```
<Button
    android:id="@+id/btnSubmit"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Submit"
    android:enabled="true" />
```

> **The text on a button is just the text attribute again, because Button extends TextView.**

THREE LOOKS

Normal, Pressed and Disabled



Disable a button with setEnabled(false) until the form is ready.

CHOOSE WISELY

Button Kinds

Button

Text on a rectangle

ImageButton

An icon, with no text

MaterialButton

Rounded, styled, ready for themes

FloatingActionButton

The round + button for a screen’s main action

QUICK RECAP

Button

- A Button is a clickable TextView
- Disable it when the input is not valid yet
- Clicking comes next

> Next up: ImageView.

WIDGET 4

ImageView

Show pictures

Fun fact: one big photo can use more memory than your whole layout, so size matters.

THE BASICS

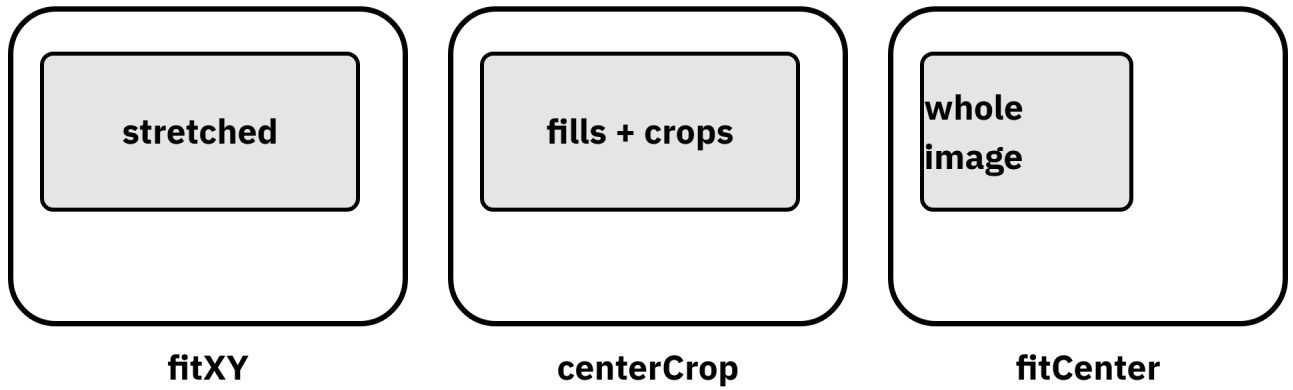
An ImageView in XML

```
<ImageView
    android:id="@+id/ivPhoto"
    android:layout_width="200dp"
    android:layout_height="200dp"
    android:src="@drawable/profile"
    android:scaleType="centerCrop"
    android:contentDescription="Profile photo" />
```

> **Put images in res/drawable. Names use lowercase letters, digits and underscores.**

FIT THE FRAME

scaleType Options



> centerCrop is the usual choice for photos. fitCenter keeps the whole image visible.

FROM JAVA

Swap Pictures at Runtime

```
ImageView ivPhoto = findViewById(R.id.ivPhoto);

ivPhoto.setImageResource(R.drawable.happy);
ivPhoto.setVisibility(View.GONE);
```

> **Accessibility tip: always write a contentDescription so screen readers can describe the picture.**

KEEP IT LIGHT

Image Tips

Right size

Do not ship a 4000-pixel photo for a 100-pixel avatar

PNG vs JPG

PNG for icons and sharp edges, JPG for photos

Vector drawables

Icons that stay sharp at any size

QUICK RECAP

ImageView

- src picks the image, scaleType decides how it fits
- setImageResource() changes it in code
- Add a contentDescription

> Next up: CheckBox, RadioButton and Switch.

WIDGETS 5 TO 7

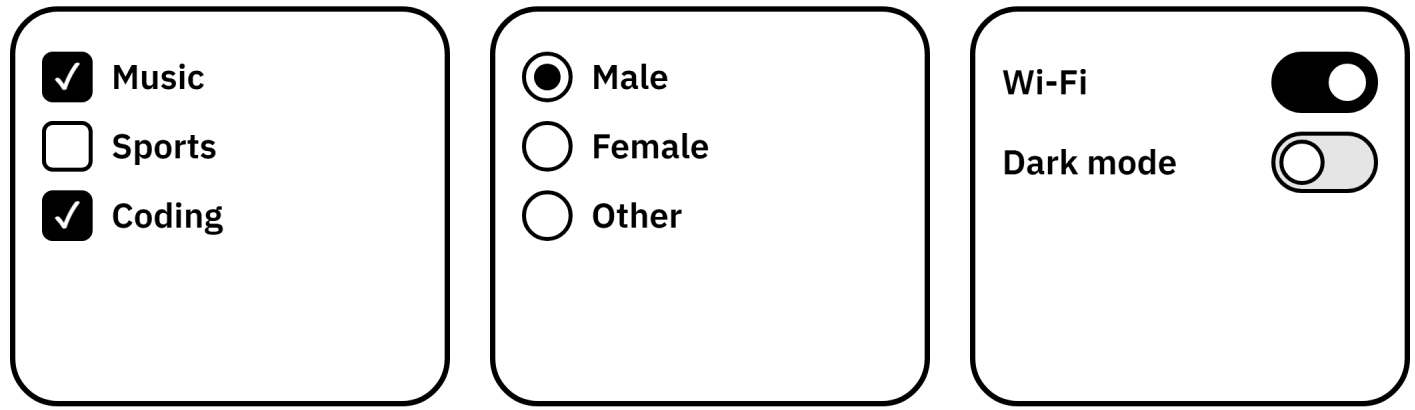
CheckBox, RadioButton & Switch

Let the user choose

Fun fact: the on/off switch on your phone settings is the very same Switch widget you will code today.

MEET THE THREE

Same Idea, Different Jobs



CheckBox: pick any

RadioButton: pick one

Switch: on or off

IN XML

CheckBox and Switch

```
<CheckBox
    android:id="@+id/cbCoding"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Coding" />

<Switch
    android:id="@+id/swWifi"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Wi-Fi" />
```

IN XML

RadioButtons Live in a RadioGroup

```
<RadioGroup
    android:id="@+id/rgGender"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">

    <RadioButton android:id="@+id/rbMale"   android:text="Male" ... />
    <RadioButton android:id="@+id/rbFemale" android:text="Female" ... />
</RadioGroup>
```

> **The group is what makes the buttons exclusive: choosing one un-chooses the others.**

IN JAVA

Reading the Choice

```
boolean coding = cbCoding.isChecked();
boolean wifiOn = swWifi.isChecked();

int id = rgGender.getCheckedRadioButtonId();
if (id == R.id.rbMale) {
    // male selected
}
```

> **isChecked() returns true or false. For a RadioGroup, ask which button id is checked.**

WHICH ONE?

Choose the Right Widget

The user needs to...	Use
Accept the terms and conditions	CheckBox
Choose a gender	RadioButton in a RadioGroup
Turn notifications on or off	Switch
Choose several hobbies	Several CheckBoxes
Choose one of two sizes	RadioGroup or Switch

QUICK RECAP

Choice Widgets

- CheckBox: any number can be ticked
- RadioButton: only one inside a RadioGroup
- Switch: an instant on or off
- All answer with isChecked()

> Next up: click listeners, the moment the app comes alive.

EVENTS

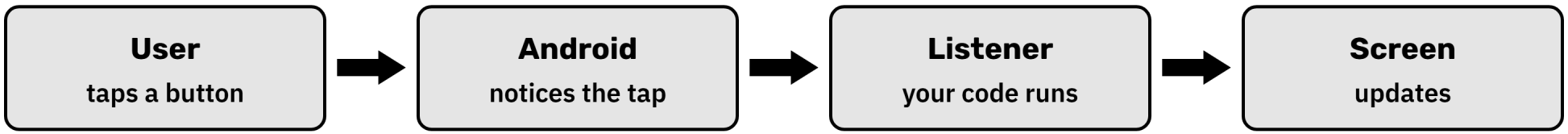
Click Listeners

Make a button do something

Fun fact: Android is event-driven. Your code sleeps until the user taps, types or scrolls.

THE IDEA

An Event Is Something the User Did



> A listener is a small piece of code you hand to a View: “call this when the user taps you”.

AN ANALOGY

Think of a Doorbell

The button

The doorbell. It does nothing by itself

The listener

The person inside who hears it

The code

What the person does: open the door

STEP 1

Give the Button an ID

```
<Button
    android:id="@+id/btnHello"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Say Hello" />
```

> **No id, no way for Java to find the button.**

STEP 2

Attach a Listener

```
Button btnHello = findViewById(R.id.btnHello);

btnHello.setOnClickListener(new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        // runs every time the button is tapped
    }
});
```

> **This is the long form. It works, but we usually write it shorter.**

SHORTER

The Lambda Way

```
btn.setOnClickListener(
  new View.OnClickListener() {
    @Override
    public void onClick(View v) {
        doIt();
    }
});
```

Long form

```
btn.setOnClickListener(v → {
    doIt();
});
```

Lambda

> **Both do the same thing. Lambdas need Java 8, which Android Studio sets up for you.**

ANOTHER WAY

android:onClick in XML

```
android:onClick="sayHello"
```

XML

```
public void sayHello(View v) {
    // runs on click
}
```

Java

> Fine for quick demos. Real apps prefer setOnClickListener because the logic stays in one place.

MANY BUTTONS

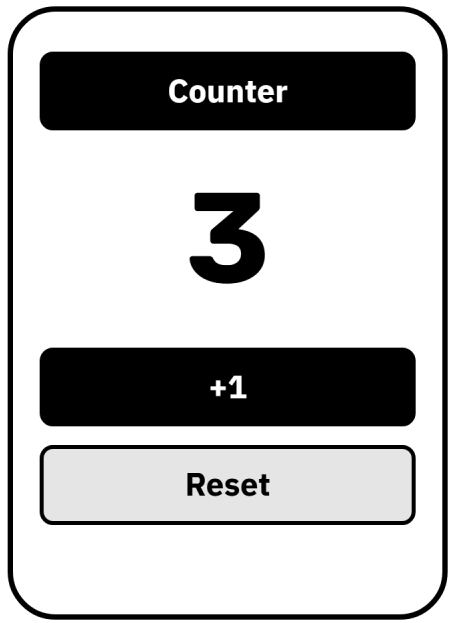
One Listener, Several Buttons

```
View.OnClickListener l = v → {
    int id = v.getId();
    if (id == R.id.btnPlus)  count++;
    else if (id == R.id.btnMinus) count--;
    tvCount.setText(String.valueOf(count));
};
btnPlus.setOnClickListener(l);
btnMinus.setOnClickListener(l);
```

> **v.getId() tells you which button was tapped.**

A TINY APP

The Counter



- Tap +1: count++ and setText()
- Tap Reset: count = 0
- The listener changes what the TextView shows

QUICK RECAP

Click Listeners

- An event is something the user does
- setOnClickListener() gives the View its reaction
- Use lambdas to keep it short
- v.getId() tells buttons apart

> Next up: reading what the user typed.

INPUT

Reading & Validating Input

Turn typing into data

Fun fact: nearly every form bug in apps comes from an empty field nobody checked.

THE GOLDEN LINE

getText().toString()

```
EditText etName = findViewById(R.id.etName);

String name = etName.getText().toString();
```

> **getText() returns an Editable, not a String. Always add .toString().**

PUT IT TOGETHER

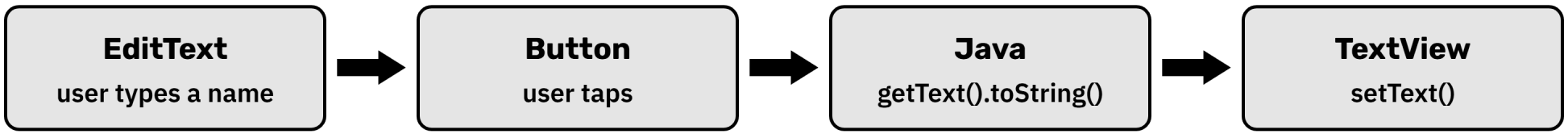
Type, Tap, See the Result

```
btnGreet.setOnClickListener(v → {
    String name = etName.getText().toString();
    tvResult.setText("Hello, " + name + "!");
});
```

> **This is the whole Greetings app. We will build it properly at the end.**

FOLLOW THE DATA

From Keyboard to Screen



NUMBERS

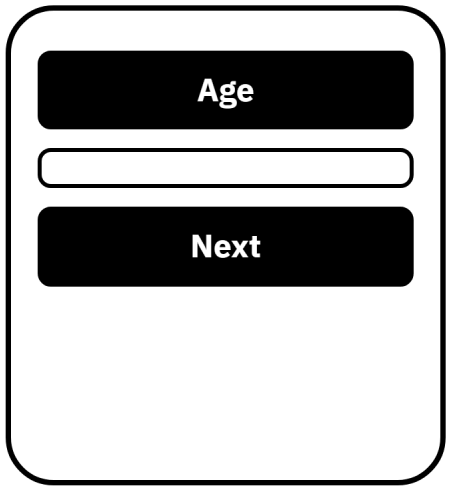
Text to Number

```
String s = etAge.getText().toString();
int age = Integer.parseInt(s);
double price = Double.parseDouble(etPrice.getText().toString());
```

> **Everything typed is text. Convert it before doing maths.**

CAREFUL

An Empty Field Crashes the App



- The age field is empty
- Integer.parseInt("") throws NumberFormatException
- The app closes with a crash
- Fix: check for empty before converting

VALIDATE

Check for Empty Input

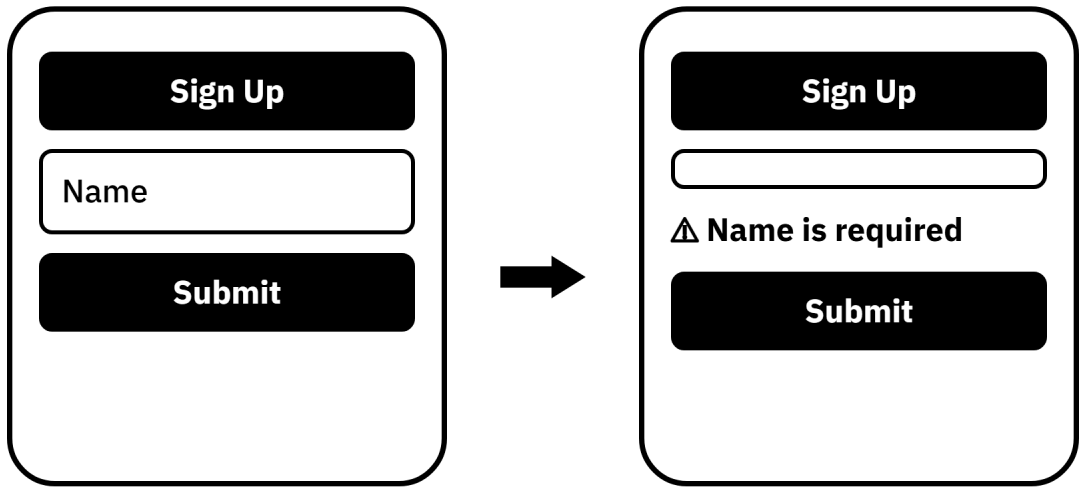
```
String name = etName.getText().toString().trim();

if (name.isEmpty()) {
    etName.setError("Name is required");
    return;
}
```

> **trim() removes spaces at the ends, so a name of only spaces still counts as empty.**

FRIENDLY FEEDBACK

setError() Shows the Message in the Field



MORE RULES

Email and Password Checks

```
String email = etEmail.getText().toString().trim();
String pass  = etPass.getText().toString();

if (!Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
    etEmail.setError("Enter a valid email");
} else if (pass.length() < 6) {
    etPass.setError("At least 6 characters");
}
```

> **Patterns.EMAIL_ADDRESS is built into Android, so you need no extra library.**

HABIT

The Validation Checklist

1. Read the text and trim() it
2. Is it empty? Show an error and stop
3. Is it the right format or length?
4. Convert to a number only after it passes
5. Then and only then use the value

QUICK RECAP

Reading & Validating

- getText().toString() reads the value
- trim() and isEmpty() catch blank input
- setError() tells the user what to fix
- Convert text to numbers after validation

> Next up: Toast and Snackbar for feedback.

FEEDBACK

Toast & Snackbar

Tell the user what happened

Fun fact: the name Toast comes from the way the message pops up, just like toast from a toaster.

TOAST

A Quick Message

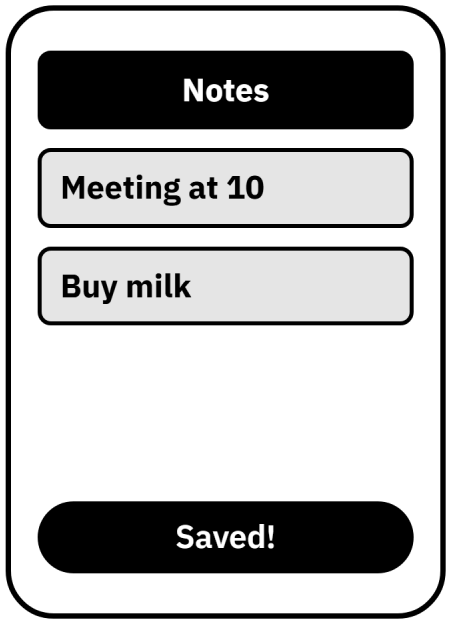
```
Toast.makeText(this, "Saved!", Toast.LENGTH_SHORT).show();
```

> **Three ingredients: context, message, duration. Do not forget .show(), the most common slip.**

```
Toast.LENGTH_SHORT   // about 2 seconds
Toast.LENGTH_LONG    // about 3.5 seconds
```

WHAT IT LOOKS LIKE

A Toast Floats Over the Screen



- Appears near the bottom
- Disappears by itself
- No buttons, and it cannot be dismissed

SNACKBAR

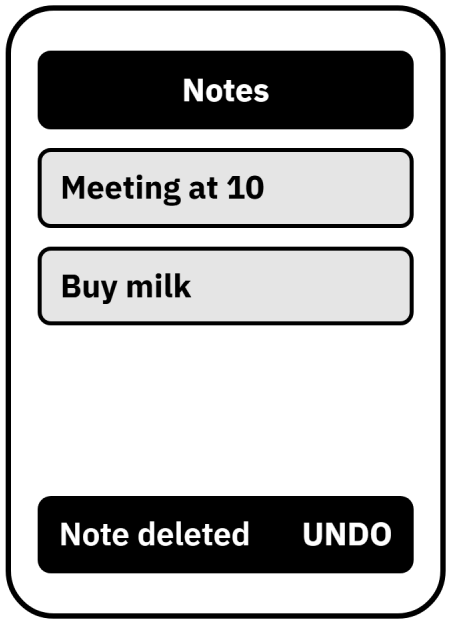
A Message With an Action

```
Snackbar.make(view, "Note deleted", Snackbar.LENGTH_LONG)
        .setAction("UNDO", v → restoreNote())
        .show();
```

> **Snackbar is part of the Material library. It needs a View to attach to.**

WHAT IT LOOKS LIKE

A Snackbar Sits at the Bottom



- Slides up from the bottom
- Can include an action such as UNDO
- Can be swiped away

COMPARE

Toast or Snackbar?

	Toast	Snackbar
Action button	No	Yes
Dismissible	No	Yes, swipe
Needs a View	No	Yes
Best for	Short confirmations	Messages the user may undo

QUICK RECAP

Toast & Snackbar

- Toast: quick, simple, no buttons
- Snackbar: can carry an action
- Remember to call .show()

> Next up: events beyond the simple click.

MORE EVENTS

Beyond the Click

Long press, typing and toggling

Fun fact: your phone keyboard’s suggestions rely on the same typing event you will use today.

LONG PRESS

setOnLongClickListener

```
btnDelete.setOnLongClickListener(v → {
    Toast.makeText(this, "Hold detected", Toast.LENGTH_SHORT).show();
    return true;
});
```

> **Return true to say “I handled this”. Return false and a normal click may fire as well.**

TOGGLING

setOnCheckedChangeListener

```
swDark.setOnCheckedChangeListener((buttonView, isChecked) -> {
    if (isChecked) {
        tvMode.setText("Dark mode on");
    } else {
        tvMode.setText("Dark mode off");
    }
});
```

> **Works on CheckBox, Switch and RadioButton, since they are all CompoundButtons.**

GROUP CHOICE

RadioGroup Listener

```
rgGender.setOnCheckedChangeListener((group, checkedId) → {
    if (checkedId == R.id.rbMale) {
        gender = "Male";
    } else if (checkedId == R.id.rbFemale) {
        gender = "Female";
    }
});
```

LIVE TYPING

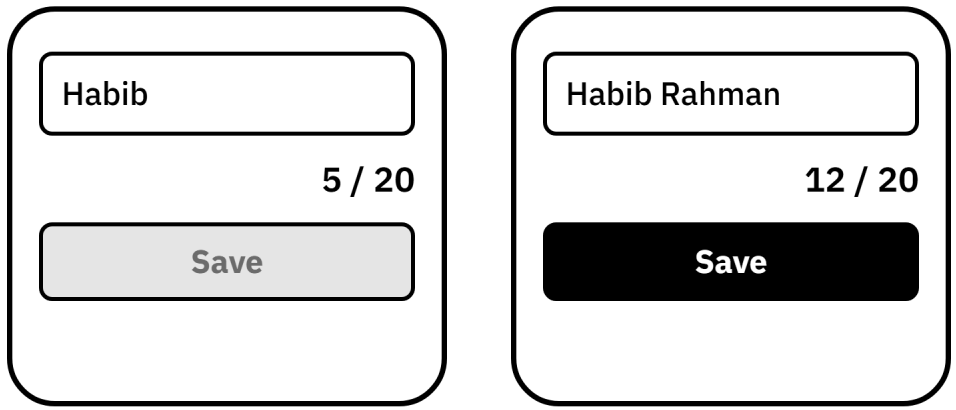
TextWatcher

```
etName.addTextChangedListener(new TextWatcher() {
    public void beforeTextChanged(CharSequence s, int a, int b, int c) {}
    public void onTextChanged(CharSequence s, int a, int b, int c) {
        tvCount.setText(s.length() + " / 20");
    }
    public void afterTextChanged(Editable e) {}
});
```

> **It fires on every keystroke. Use it for counters, live search and enabling buttons.**

LIVE FEEDBACK

A Character Counter



Typing… Keep typing

> The Save button turns on once the name is long enough.

CHEAT SHEET

Which Listener for Which Widget?

Event	Listener
Tap	setOnClickListener
Long press	setOnLongClickListener
Switch, checkbox toggled	setOnCheckedChangeListener
Text changed	addTextChangedListener
Keyboard Done key	setOnEditorActionListener

QUICK RECAP

Other Events

- Long click, checked-changed and text changed all follow the listener pattern
- TextWatcher reacts to every keystroke
- Pick the listener that matches the user’s action

> Next up: ViewBinding, a safer way than findViewById.

CLEANER CODE

ViewBinding

Say goodbye to findViewById

Fun fact: findViewById returns a plain View, so a typo’s id can crash your app only when it runs.

THE PAIN

Too Many findViewById Lines

```
EditText etName  = findViewById(R.id.etName);
EditText etEmail = findViewById(R.id.etEmail);
EditText etPass  = findViewById(R.id.etPass);
Button   btnSave = findViewById(R.id.btnSave);
TextView tvInfo  = findViewById(R.id.tvInfo);
// ...and so on for every widget
```

> **It is boring, easy to mistype, and gives no warning if the id is wrong.**

STEP 1

Turn It On

```
android {
    ...
    buildFeatures {
        viewBinding true
    }
}
```

> **Add it to the app-level build.gradle, then press Sync Now.**

STEP 2

Use the Generated Class

```
private ActivityMainBinding binding;

@Override
protected void onCreate(Bundle b) {
    super.onCreate(b);
    binding = ActivityMainBinding.inflate(getLayoutInflater());
    setContentView(binding.getRoot());

    binding.btnGreet.setOnClickListener(v →
        binding.tvResult.setText("Hello, " + binding.etName.getText() + "!"));
}
```

> **activity_main.xml becomes ActivityMainBinding. Every id becomes a field.**

COMPARE

findViewById vs ViewBinding

	findViewById	ViewBinding
Typos in ids	Crash at runtime	Error while compiling
Wrong type	Possible crash	Impossible
Lines of code	Many	Few
Setup	None	One line in Gradle

QUICK RECAP

ViewBinding

- Enable viewBinding in Gradle
- Inflate ActivityMainBinding and use binding.yourId
- Fewer lines and safer code

> Next up: build the Greetings app with everything we learned.

PUT IT TOGETHER

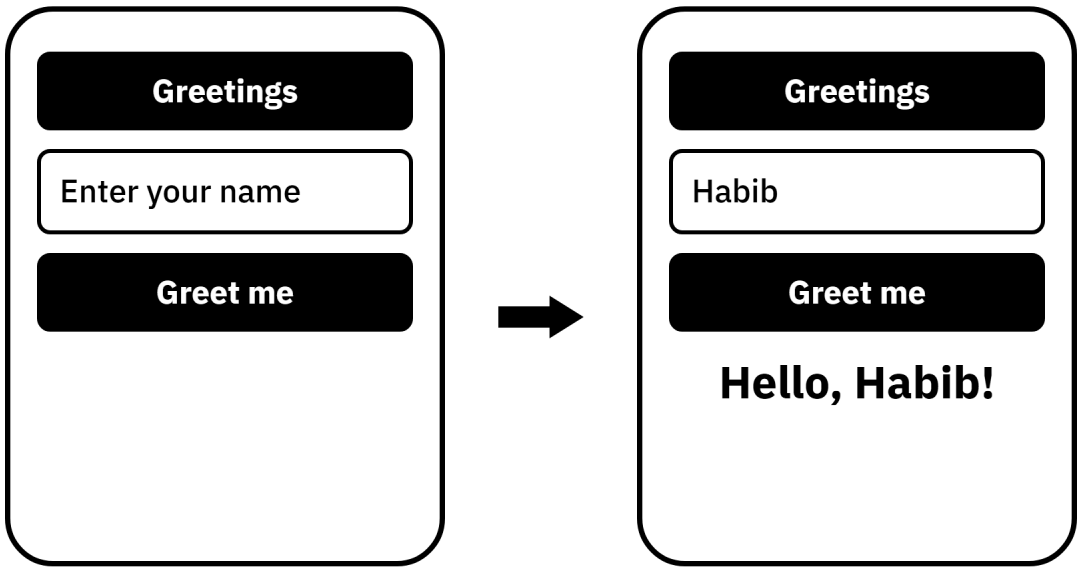
The Greetings App

One screen, five ideas

Fun fact: a lot of real apps are just a few input widgets and one button wired up well.

THE GOAL

What We Are Building



Before

After tapping

PLAN FIRST

Widgets and Events We Need

1. TextView: shows the result
2. EditText: for the name
3. Button: Greet me
4. Click listener: read the name, validate it, show the greeting
5. Toast: confirm that it worked

STEP 1

The Layout

```
<LinearLayout ... android:orientation="vertical" android:padding="24dp">

    <EditText android:id="@+id/etName"
        android:layout_width="match_parent" android:layout_height="wrap_content"
        android:hint="Enter your name" android:inputType="textPersonName" />

    <Button android:id="@+id/btnGreet"
        android:layout_width="match_parent" android:layout_height="wrap_content"
        android:text="Greet me" />

    <TextView android:id="@+id/tvResult"
        android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:textSize="28sp" />
</LinearLayout>
```

STEP 2

The Activity

```
btnGreet.setOnClickListener(v → {
    String name = etName.getText().toString().trim();

    if (name.isEmpty()) {
        etName.setError("Please enter your name");
        return;
    }

    tvResult.setText("Hello, " + name + "!");
    Toast.makeText(this, "Greeted " + name, Toast.LENGTH_SHORT).show();
});
```

LOOK BACK

Which Idea Did We Use?

Idea	Where
EditText + inputType	The name field
Click listener	btnGreet.setOnClickListener
Reading input	getText().toString().trim()
Validation	isEmpty() + setError()
Feedback	setText() and Toast

MAKE IT YOUR OWN

Ideas to Extend the App

Add a greeting style

RadioButtons: formal or friendly

Add a Switch

Shout the greeting in capital letters

Show a picture

ImageView changes with the choice

Clear button

Resets the name and the greeting

QUICK RECAP

The Greetings App

- Small apps combine several small ideas
- Read, validate, then show
- Add the features one at a time and test each

> Next up: a quick quiz and the common mistakes.

TEST YOURSELF

Which Widget or Listener?

1. The user types an email address
2. The user picks exactly one payment method
3. The user turns on notifications
4. You want to count characters while the user types
5. You want to show a short message that disappears by itself

> Hint: EditText, RadioGroup, Switch, TextWatcher or Toast

THE ANSWERS

Solved

Situation	Answer
Types an email address	EditText with textEmailAddress
Exactly one payment method	RadioGroup
Turns on notifications	Switch
Count characters live	TextWatcher
Short disappearing message	Toast

LEARN FROM THESE NOW

Common Mistakes

Forgetting .show()

The Toast never appears

Missing .toString()

Comparing an Editable with a String gives wrong results

Parsing empty text

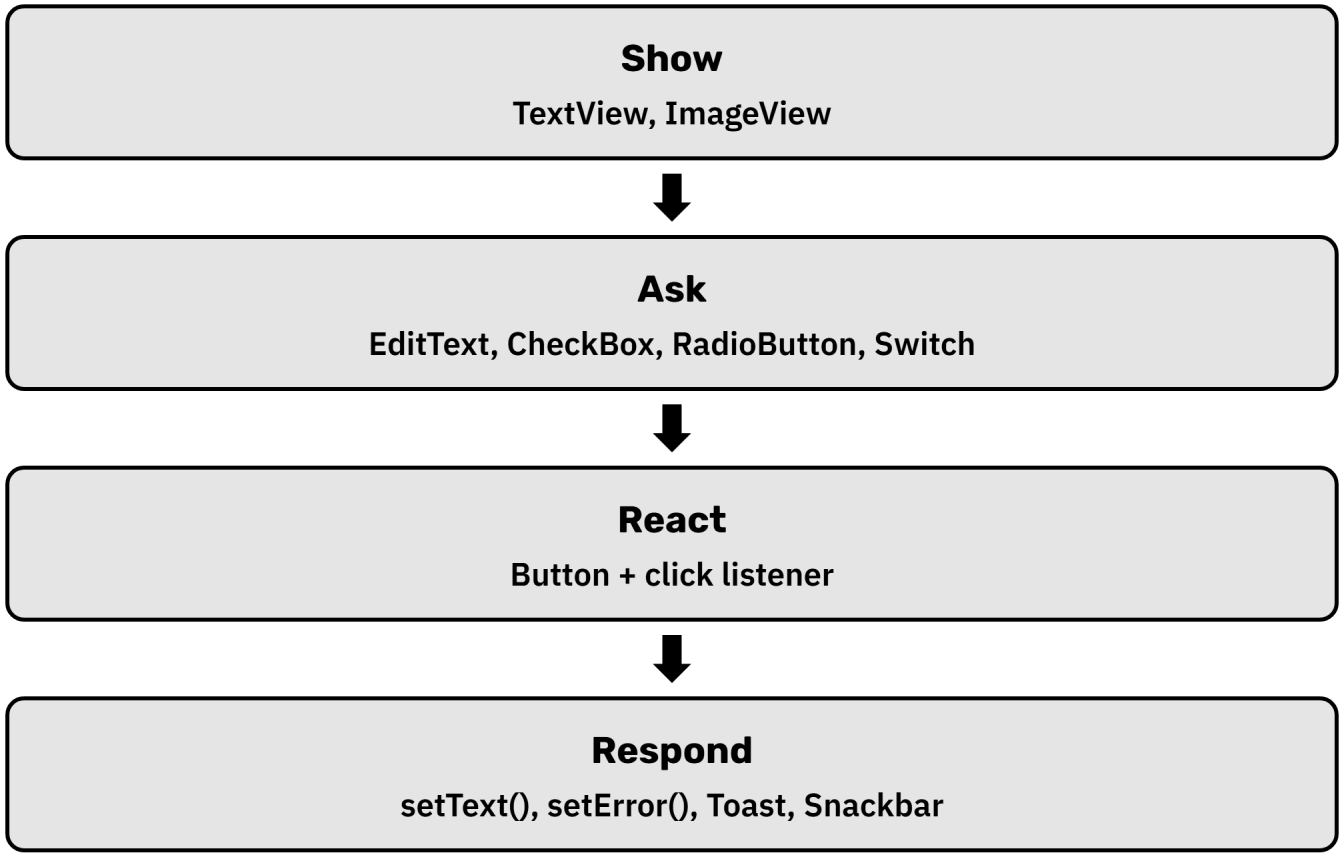
Integer.parseInt("") crashes the app

Same id twice

findViewById returns the wrong view

ONE SLIDE TO REMEMBER

The Whole Pattern



KEEP THIS ONE

Quick Reference

To do this	Write this
Find a view	findViewById(R.id.x) or binding.x
Read typed text	et.getText().toString()
Change text	tv.setText("...")
React to a tap	btn.setOnClickListener(v -> {...})
Check a box	cb.isChecked()
Show a message	Toast.makeText(...).show()

Questions?

TextView | EditText | Button | ImageView | Listeners

Ekramul Haque Amin

Lecturer, AI & Data Science · Mobile Application and Development (0613-409)